

Trabajo práctico Organización de Computadoras

Juan Manuel Antuña Isaías Cheij Tomás Miranda
Hilario Yáñez Bertola

Junio de 2026

1 Primera parte: Un algoritmo de multiplicación microprogramado

En esta primera parte vamos a desarrollar un (o un par de) algoritmo(s) de multiplicación basados en microinstrucciones. Puntualmente, vamos a implementar una multiplicación para números enteros de 8 bits, representados en complemento a la base (two's complement). Usamos como base el procesador desarrollado durante la materia en CPUSim.

1.1 Usando lo que ya tenemos: una solución inmediata

La primera solución que pensamos, la cual nos parece al mismo tiempo la más intuitiva, es usar los registros que ya teníamos disponibles para replicar el algoritmo. Este planteaba, para 8 bits en particular, el uso de tres registros distintos de 8 bits (considerando que el resultado puede necesitar una representación de hasta 16 bits):

- Un registro para el multiplicando.
- Uno para el multiplicador.
- Un acumulador, el cual es inicializado en 0 y se interpreta como la parte superior de un registro de 16 bits, conformado junto con el registro del multiplicador.

Utilizando estos tres registros, el algoritmo era el siguiente:

1. Ver el bit menos significativo del multiplicador. Si es un 0, saltar el siguiente paso
2. Sumar al valor actual del acumulador el del multiplicando, y guardarlo en el acumulador
3. Realizar un shift aritmético del registro de 16 bits [ACC — MULTIPLICADOR]
4. Si hubo overflow al hacer la suma, cambiar la extensión del signo por el que indica el bit de carry
5. Repetir 8 veces desde el paso (1)

Con esto en mente, para simular el formato de registros con nuestro procesador de CPUSim podemos pensar en hacer lo siguiente:

- Usar los 8 bits superiores del registro B como multiplicando
- Usar los 8 bits inferiores de A como multiplicador
- Usar los 8 bits superiores de A como acumulador

Esto nos permite usar la microinstrucción de adición provista por CPUSim. Como observación, el formato que planteamos requiere que los 8 bits inferiores del registro B sean 0's, para no alterar el valor del multiplicador a la hora de hacer las sumas parciales.

Con esto en mente, se puede replicar el algoritmo mediante el uso de microinstrucciones, como se ve a continuación:

1.2 Una segunda solución: Cómo hacer una unidad aritmética escalable

Frente a los problemas planteados de repetición de microinstrucciones y ajuste del formato a los registros presentes, pensamos cómo podemos hacer una solución que resulte más elegante y escalable para implementar aritmética con representaciones que difieran de la arquitectura del procesador. Creemos, por motivos que mencionaremos más adelante, que una solución óptima para los problemas planteados es el diseño e implementación de una unidad independiente (un conjunto de registros) dedicada exclusivamente a hacer operaciones aritméticas. A esta la llamamos AR y consta de registros descritos a continuación:

- AR-A: Un registro del tamaño asignado a la unidad sobre el cual guardaremos operandos y usaremos para realizar operaciones básicas.
- AR-B: Similar a AR-A. Particularmente, lo usaremos para almacenar el multiplicador durante la multiplicación.
- AR-C: Un registro del tamaño adecuado con la finalidad principal de ser un contador. Esto permitirá iterar secuencias de instrucciones, facilitando la escritura y lectura de instrucciones microprogramadas.
- AR-D: El último registro de nuestra unidad. Su finalidad será, principalmente, ser usado como acumulador para distintas operaciones.

Este diseño nos permitirá implementar el algoritmo de la multiplicación de forma iterativa (usando el registro AR-C como contador), facilitando la lectura y escritura del microprograma que lo define. Al mismo tiempo, esto hace que la unidad sea fácilmente extensible a aritméticas de otros tamaños, independizándose de la arquitectura de nuestro procesador y permitiendo implementar operaciones más complejas. El detalle de la implementación se encuentra en el archivo adjunto de CPUSim.

2 Segunda parte: Sumando matrices en assembly

Pasemos ahora a la segunda parte del trabajo. El planteo es el siguiente:

Diseñar una subrutina en assembly que reciba cuatro parámetros: tres matrices y un entero positivo (el tamaño de las anteriores), la cual sume las dos primeras, guardando el resultado en la tercera. Además, escribir un programa en C que use la subrutina creada.

Primero vamos a abordar nuestra solución al programa de assembly. El perfil de la función es: `add_matrices (int* m, int* n, int* res, short s)`. Acá vamos a hacer algunas aclaraciones sobre los parámetros planteados:

- Los primeros tres son punteros (direcciones de memoria) a lugares que contienen un número entero de 32 bits. `m`, `n`, `res` son la primera, segunda y matriz resultado, respectivamente.
- El último es la dimensión de las matrices a sumar. Véase que es de tipo `short`, el cual representa un entero de 16 bits. Nuestra elección se debe a que, como vimos en la primera parte, una multiplicación entre dos números de n bits puede necesitar hasta $2n$ bits para su representación. Como vamos a trabajar con un assembly en 32 bits, representamos el tamaño en 16 bits para que su cuadrado (la cantidad de elementos en las matrices) no exceda los 32 bits.

Con esto en mente, podemos replicar el algoritmo de suma de matrices, como se ve en el archivo de NASM adjunto.

Una vez implementada la subrutina, solamente nos queda vincularla a un programa de C para su ejecución. El código de C resulta bastante simple, solamente teniendo en cuenta que la definición de la función `add_matrices` debe hacerse mediante el uso de la palabra reservada `extern`, indicando que su implementación será externa al programa actual. Pongamos el foco en la forma en la que compilaremos y vincularemos los programas. Mediante el ensamblador de NASM, podemos transformar nuestro código de `sum.asm` en código objeto `sum.o`. Este aún no es ejecutable, ya que debe ser linkeado con el resto de archivos propios del proyecto. A modo de observación, al momento de ensamblaje debemos usar la flag `-f elf32`, para indicar a NASM que queremos usar el modo de 32 bits. Veamos ahora cómo compilar el programa de C para obtener un ejecutable. En este caso, usaremos el compilador GCC en su versión 15.2.1. Vamos a realizar dos tareas. Primero compilamos el archivo de C sin linkearlo, dando como resultado un archivo `main.o`. Esto se puede lograr usando GCC con la flag `-c`, lo cual generará un archivo de código objeto listo para ser linkeado. Además, debemos aclarar que el modo de compilación es de 32 bits mediante la flag `-m32`. Finalmente, podemos usar GCC mismo como linker de los archivos de código objeto, generando un archivo ya ejecutable. Usamos la flag `-o executable` para generar el ejecutable final bajo el nombre de `executable`.

Todo este proceso puede ser automatizado mediante un archivo Makefile, el cual haciendo uso de la herramienta `make` del proyecto GNU, permitiendo agilizar el proceso de

scripto en el párrafo anterior, así como agregar una rutina de limpieza (clean), que elimina el ejecutable y toda la basura generada.